SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

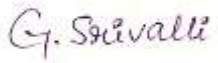
Weightage: 10%

QUESTIONS: **Total Marks: 50**

Code of Conduct:

I _G. Srivalli (192365014)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



A3- Short Answer Test

Question 1 (5 Marks)
Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.
Question 2 (5 Marks)
Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.
Question 3 (5 Marks)
Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.
Question 4 (5 Marks)
Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.
Question 5 (5 Marks)
Responsive Layout: A college website must display properly on desktops, tablets, and mobile phones. How would you use CSS media queries and flexible layouts to achieve responsiveness?
Question 6(5 Marks)
CSS Styling: An online shopping website has plain product cards. How would you apply CSS colors, borders, shadows, and spacing to improve its visual appearance?
Question 7 (5 Marks)
Navigation Menu: A website navigation menu overlaps its content on smaller screens. How would you use CSS layout techniques to correct the problem?
Question 8 (5 Marks)
JavaScript Validation: A registration form accepts empty fields. How would you use JavaScript to validate the form before submission?

Question 9 (5 Marks)
Interactive Button: An e-learning website requires a button that displays a welcome message when clicked. How would you implement this functionality using JavaScript?
Question 104 (5 Marks)
. Dynamic Content: A shopping website needs to update the total price when a user changes the quantity of a product. How would you use JavaScript to calculate and display the updated price?

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1: Responsive Web Design Principles and Implementation

Concept & Importance: Responsive Web Design (RWD) is an approach where a website dynamically adapts its layout, images, and UI components to fit various screen sizes and resolutions (mobile, tablet, desktop). It eliminates horizontal scrolling, improves readability, lowers maintenance costs compared to maintaining separate device-specific sites, and optimizes SEO rankings.

Key CSS Techniques

- **Fluid Grids/Units**: Using relative units like %, vw, vh, rem, and em instead of rigid px dimensions.
- **Flexible Images**: Setting max-width: 100%; height: auto; to keep media within their parent containers.
- **Media Queries**: @media rules that apply conditional styling based on viewport breakpoints (e.g., min-width or max-width).

CSS Flexbox vs. CSS Grid:

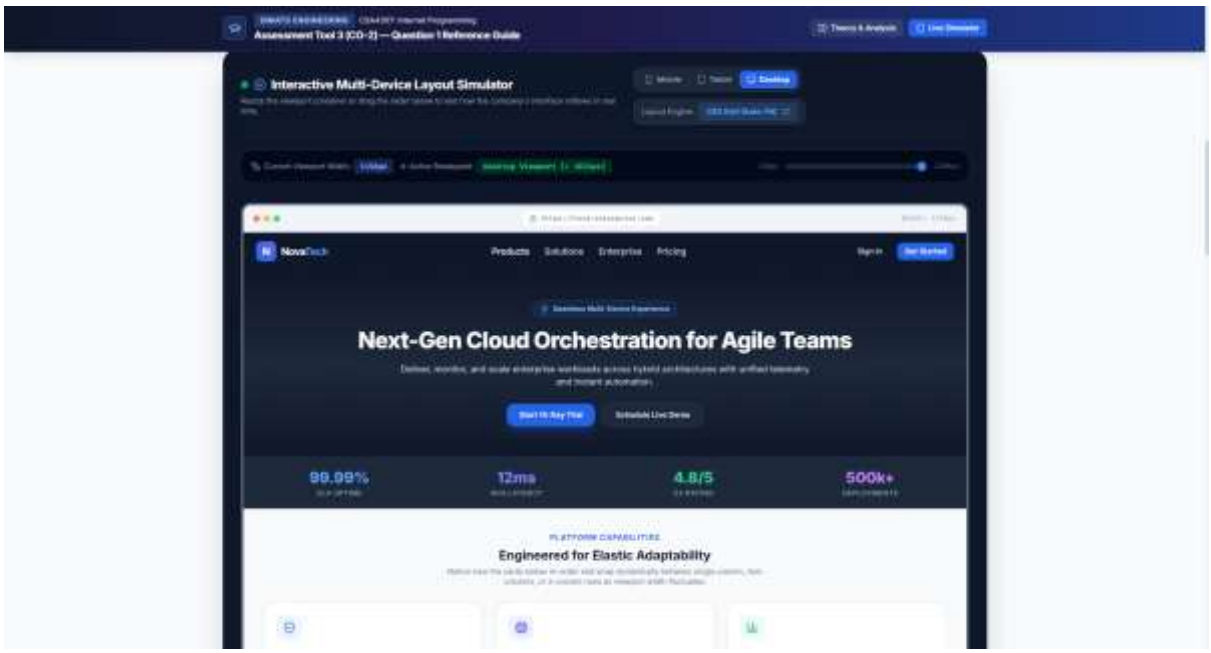
- **Flexbox (One-Dimensional)**: Best suited for distributing elements along a single row or column (e.g., navigation bars, form controls) using properties like flex-wrap: wrap and justify-content: space-between.
- **CSS Grid (Two-Dimensional)**: Ideal for page scaffolding across rows and columns simultaneously. The pattern grid-template-columns: repeat(auto-fit, minmax(250px, 1fr)); automatically reflows columns across any viewport width without excessive media queries.

Code Example:
HTML

```
<div class="card-container">
  <div class="card">Item 1</div>
```

```
  <div class="card">Item 2</div>
  <div class="card">Item 3</div>
</div>
<style>
  .card-container {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
    gap: 16px;
    padding: 16px;
  }
  .card {
    background: #f4f4f4;
    padding: 20px;
    border-radius: 8px;
  }
</style>
```

UX Improvements: Implement mobile-first media query breakpoints (min-width), set appropriate touch targets (minimum 44×44 px), and optimize typography using clamp scaling (e.g., font-size: clamp(1rem, 2.5vw, 1.5rem);).



Question 2: Client-Side Validation and Usability

Concept & Purpose: Client-side validation inspects user input in the browser before dispatching the payload to the server. Its primary purpose is to provide immediate feedback to the user, reduce latency, and prevent unnecessary server load from malformed data.

JavaScript Event Handling: Form events such as input or blur offer real-time field-level checks, while the submit event intercepts the submission cycle via event.preventDefault() to halt submission if validation criteria fail.

Role of Regular Expressions (RegEx): RegEx defines structural pattern-matching rules for strings (e.g., verifying that an email follows standard username, domain, and TLD conventions or that a phone number matches local numerical formats).

Code Example:

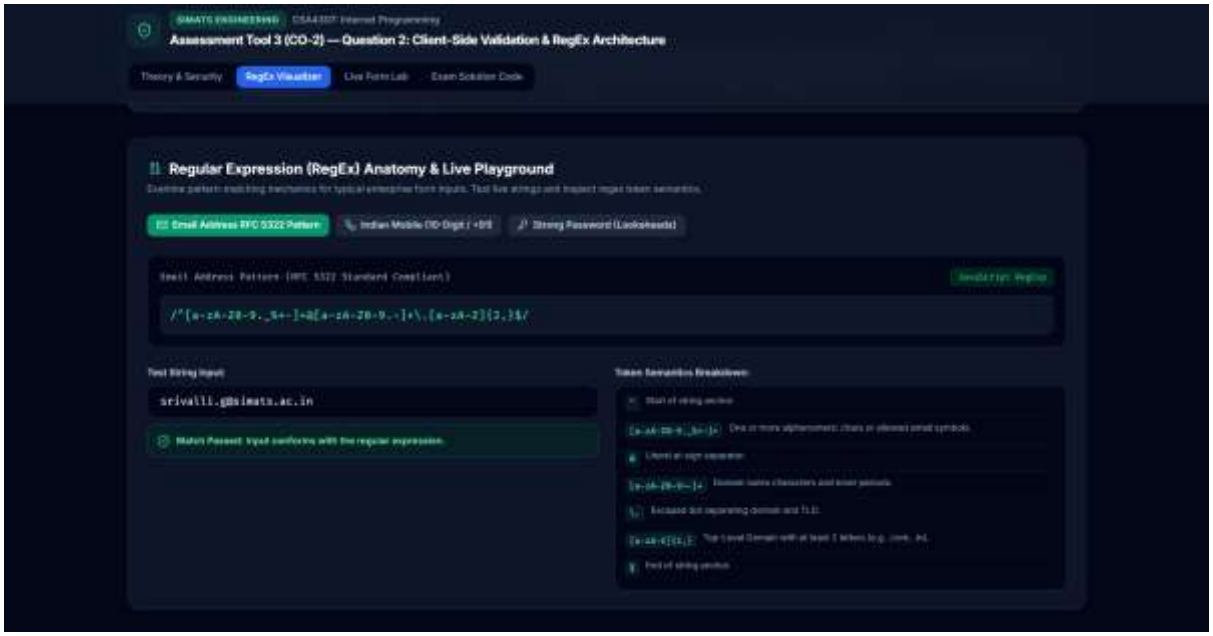
HTML

```
<form id="userForm">
  <input type="text" id="email" placeholder="Enter email" />
  <span id="error" style="color: red;"></span>
  <button type="submit">Submit</button>
</form
<script>
  const form = document.getElementById('userForm');
  const emailInput = document.getElementById('email');
  const errorSpan = document.getElementById('error');
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  form.addEventListener('submit', (e) => {
    if (!emailPattern.test(emailInput.value.trim())) {
      e.preventDefault();
      errorSpan.textContent = 'Please enter a valid email address.';
    } else {
      errorSpan.textContent = '';
```

```
    }
  });
</script>
```

Usability & Security Enhancements:

- Usability: Use inline real-time feedback (input listener) and display clear error messages next to the offending input field.
- Security: Client-side checks can be bypassed (e.g., by disabling JavaScript or using API tools like Postman/cURL); server-side validation and data sanitization are always mandatory for defense-in-depth.



Question 3: CSS Box Model, Positioning, and Layout Stability

CSS Box Model Components:

- **Content**: The central area where text, images, or child elements render.
- **Padding**: Transparent space between the content and the border.
- **Border**: The stroke/line wrapping around the padding and content.
- **Margin**: Clear spacing outside the border, separating the element from neighbors.

Positioning Techniques:

- static: Standard document flow.
- relative: Positioned relative to normal flow; offset using top, left, etc.
- absolute: Removed from normal document flow; positioned relative to the nearest positioned ancestor.
- fixed: Removed from flow; anchored to the viewport.
- sticky: Toggles between relative and fixed depending on scroll position.

Causes of Layout Breaking:

- Default box-sizing: content-box, which adds padding and borders on top of the specified width, causing elements to exceed 100% width.
- Hardcoded pixel widths (e.g., width: 1200px) overflowing mobile viewports.
- Mismanaged floating elements (float) without clearfixes or over-reliance on position: absolute on responsive containers.

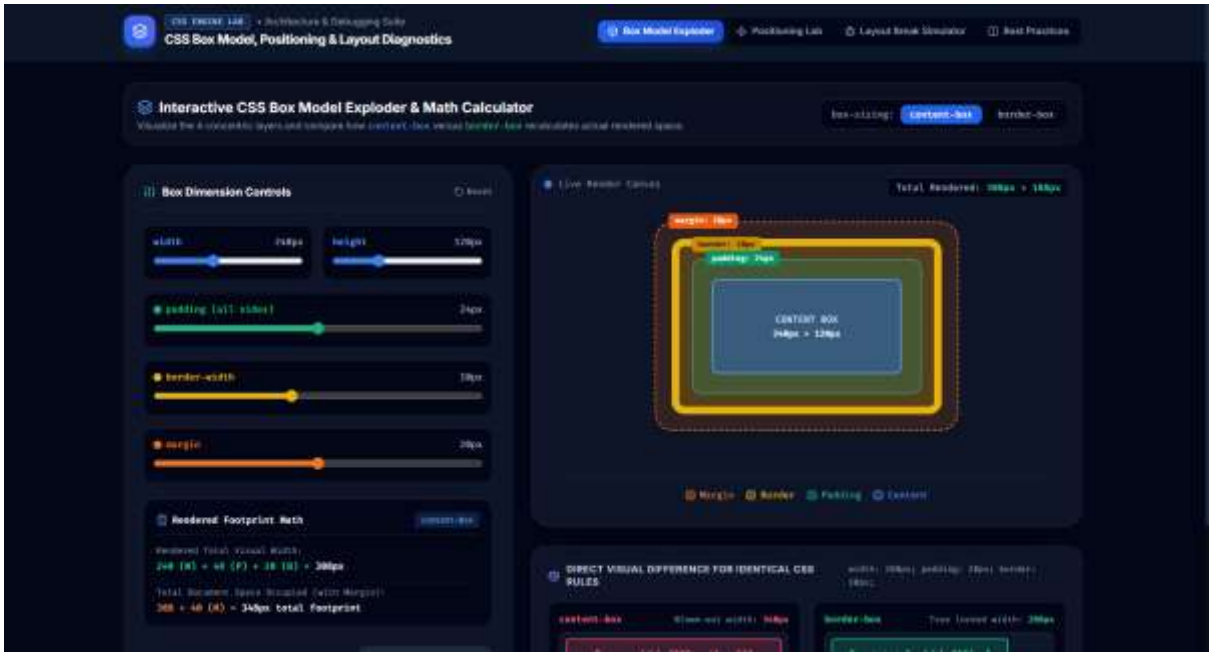
Resolution via Media Queries:

CSS

```
@media (max-width: 768px) {
  .sidebar, .main-content {
    width: 100%;
    float: none;
  }
}
```

Best Practices:

- Apply universal border-box sizing: *, *::before, *::after { box-sizing: border-box; }.
- Use modern layout engines (Flexbox/Grid) instead of legacy float patterns.
- Use relative constraints like max-width: 100% rather than fixed absolute widths.



Question 4: Document Object Model (DOM) and Dynamic Manipulation

DOM Concept & Role: The DOM represents an HTML document as a structured, hierarchical tree of nodes and objects in memory. It serves as the programmatic interface allowing scripting languages like JavaScript to read, traverse, insert, remove, or modify elements, attributes, and styles dynamically.

JavaScript Manipulation Mechanisms:

- Targeting: document.querySelector(), document.getElementById().
- Modifying: element.textContent, element.innerHTML, element.setAttribute().
- Creating & Appending: document.createElement(), parent.appendChild(), element.remove().

Code Example:

HTML

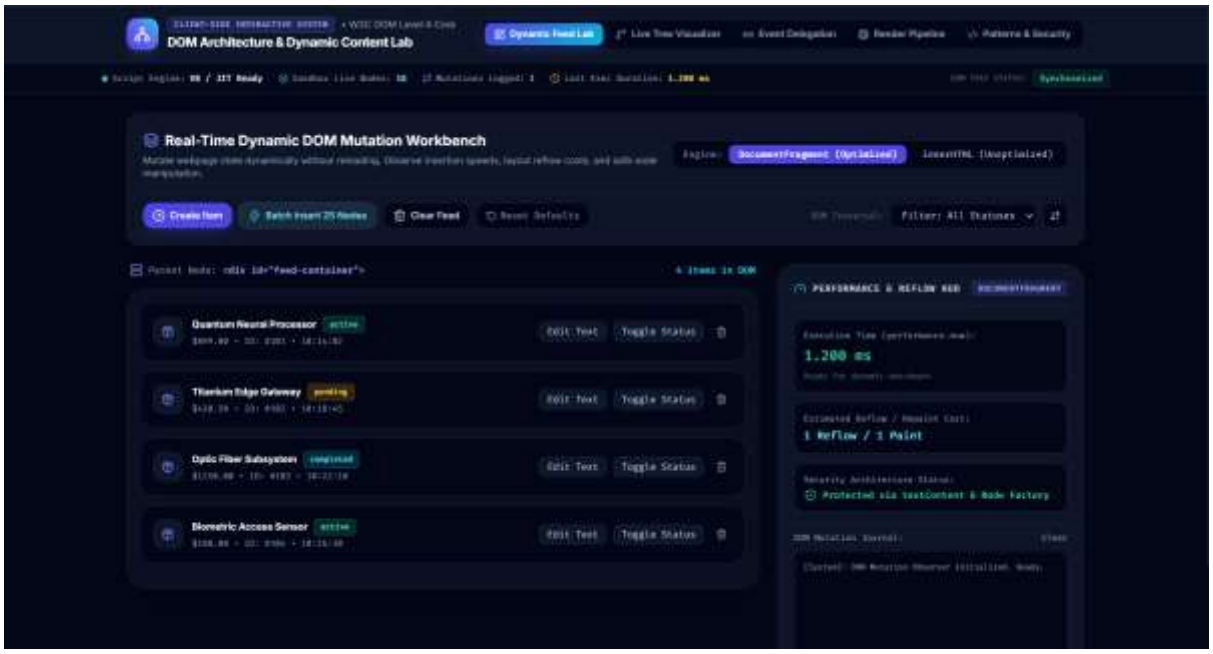
```
<ul id="taskList">
  <li>Review Syllabus</li>
</ul>
<button id="addBtn">Add Task</button>

<script>
  const button = document.getElementById('addBtn');
  const taskList = document.getElementById('taskList');

  button.addEventListener('click', () => {
    const newLi = document.createElement('li');
    newLi.textContent = 'Complete Lab Assignment';
    taskList.appendChild(newLi);
  });
</script>
```

Performance & UX Improvements:

- Use DocumentFragment when inserting multiple elements to minimize layout recalculations (reflows and repaints).
- Prefer textContent over innerHTML when handling non-HTML text to mitigate Cross-Site Scripting (XSS) risks.
- Use event delegation (attaching listeners to a common ancestor) to manage dynamically created child nodes efficiently.



Question 5: College Website Responsive Layout Implementation

Mobile-First CSS Implementation:

HTML

```
<div class="college-layout">
  <header class="header">College Header & Nav</header>
  <main class="content">Course Details & Announcements</main>
  <aside class="sidebar">Events & Deadlines</aside>
  <footer class="footer">Footer Info</footer>
</div>
<style>
  /* Base: Mobile Viewport (Single Column) */
  .college-layout {
    display: flex;
    flex-direction: column;
    gap: 16px;
    padding: 12px;
  }
  /* Tablet Breakpoint (Two Columns) */
  @media (min-width: 600px) {
    .college-layout {
      display: grid;
      grid-template-columns: 2fr 1fr;
      grid-template-areas:
        "header  header"
        "content sidebar"
        "footer  footer";
    }
    .header { grid-area: header; }
    .content { grid-area: content; }
    .sidebar { grid-area: sidebar; }
    .footer { grid-area: footer; }
```

```
  }
  /* Desktop Breakpoint (Expanded Multi-Column) */
  @media (min-width: 1024px) {
    .college-layout {
      max-width: 1200px;
      margin: 0 auto;
      grid-template-columns: 3fr 1fr;
    }
  }
</style>
```

Summary: The mobile layout stacks blocks vertically using standard column flex. At min-width: 600px and min-width: 1024px, CSS Grid positions the sidebar alongside main content, maximizing screen real estate while keeping markup intact.



Question 6: Enhancing Product Cards with CSS Styling

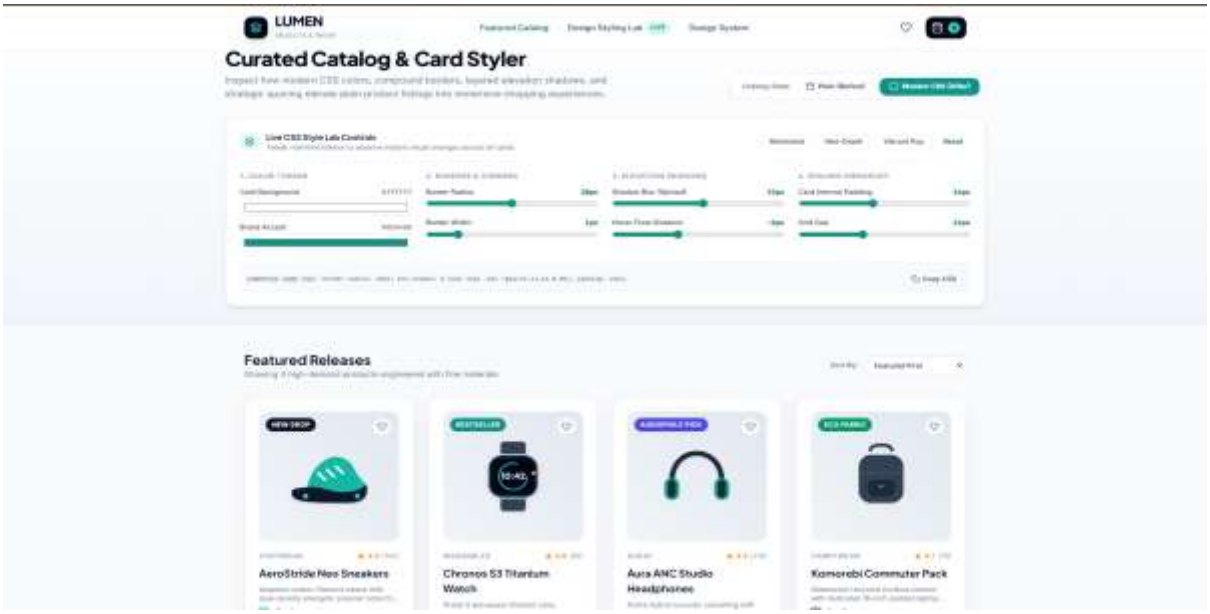
Styling Implementation:

HTML

```
<div class="product-card">
  <img src="placeholder.jpg" alt="Item" class="product-img">
  <div class="product-info">
    <h3 class="product-title">Wireless Headphones</h3>
    <p class="product-price">$99.99</p>
    <button class="btn-buy">Add to Cart</button>
  </div>
</div>
<style>
  .product-card {
    width: 280px;
    background-color: #ffffff;
    border: 1px solid #e0e0e0;
    border-radius: 12px;
    overflow: hidden;
    /* Soft elevation shadow */
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
    transition: transform 0.2s ease, box-shadow 0.2s ease;
  }
  .product-card:hover {
    transform: translateY(-4px);
    box-shadow: 0 8px 24px rgba(0, 0, 0, 0.14);
  }
  .product-info {
    padding: 18px; /* Internal spacing */
  }
  .product-title {
    margin: 0 0 8px 0;
```

```
    font-size: 1.1rem;
    color: #222222;
  }
  .product-price {
    margin: 0 0 16px 0;
    color: #0b7285;
    font-weight: bold;
  }
  .btn-buy {
    width: 100%;
    padding: 10px;
    background-color: #0c8599;
    color: #ffffff;
    border: none;
    border-radius: 6px;
    cursor: pointer;
  }
</style>
```

Styling Applied: Rounded borders (border-radius), subtle resting and hover drop shadows (box-shadow), balanced padding (padding), and color contrast between header, pricing, and the action button.



Question 7: Preventing Navigation Menu Overlap on Small Screens

Problem Cause: Navbars often overlap content when elements have fixed heights, absolute positioning, or lack wrapping rules on small viewports.

CSS Fix (Flexbox Wrap & Hamburger Toggle Layout):

HTML

```
<header class="navbar">
  <div class="logo">Brand</div>
  <nav class="nav-links">
    <a href="#">Home</a>
    <a href="#">Courses</a>
    <a href="#">Admissions</a>
    <a href="#">Contact</a>
  </nav>
</header>
<style>
  .navbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 16px 24px;
    background-color: #1a1a1a;
  }

  .nav-links {
    display: flex;
    gap: 20px;
  }
  /* Responsive breakdown to prevent overlap */
  @media (max-width: 768px) {
    .navbar {
```

```
    flex-direction: column;
    align-items: flex-start;
  }
  .nav-links {
    flex-direction: column;
    width: 100%;
    gap: 12px;
    margin-top: 12px;
    display: none; /* Can be toggled open via a .show class */
  }
  .nav-links.active {
    display: flex;
  }
 }
</style>
```

Summary: Switching from a rigid horizontal row to flex-direction: column underneath 768px ensures the menu links stack vertically or collapse into a toggleable drawer rather than colliding with the brand header or adjacent content.



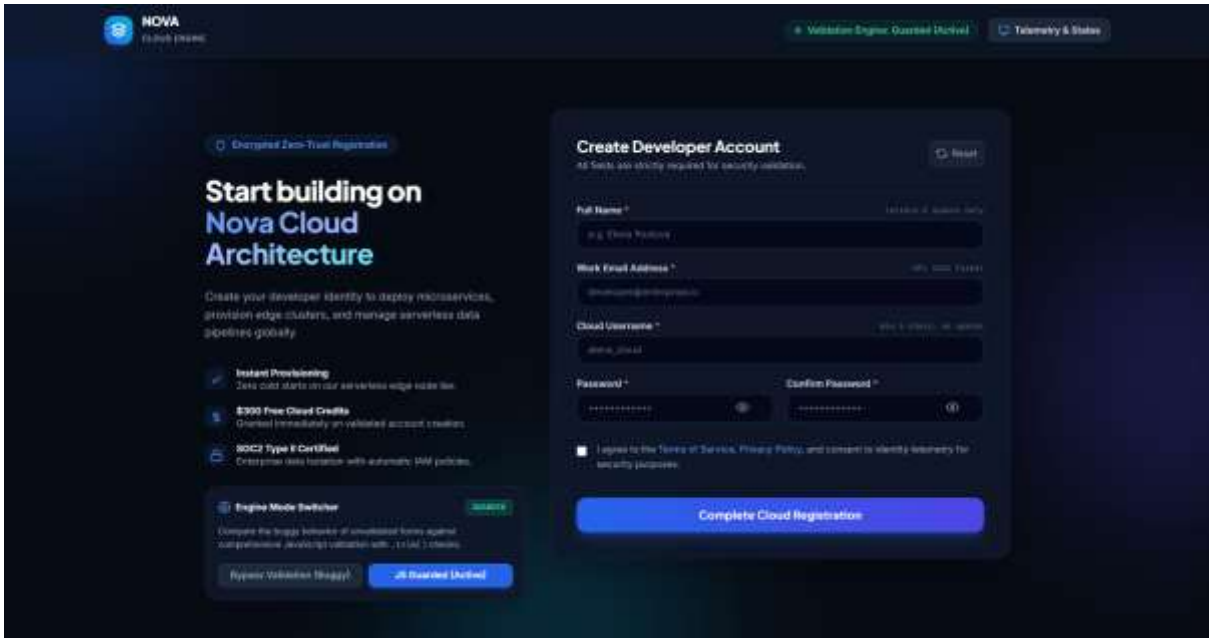
Question 8: Form Validation for Empty Fields Using JavaScript

Validation Script:

HTML

```
<form id="regForm">
  <div>
    <label for="username">Username:</label>
    <input type="text" id="username">
    <span id="userError" style="color: red;"></span>
  </div>
  <button type="submit">Register</button>
</form>
<script>
  const form = document.getElementById('regForm');
  const usernameInput = document.getElementById('username');
  const userError = document.getElementById('userError');
  form.addEventListener('submit', function(event) {
    // Clear previous errors
    userError.textContent = "";
    // Check for empty or whitespace-only input
    if (usernameInput.value.trim() === "") {
      event.preventDefault(); // Stop form submission
      userError.textContent = "Field cannot be empty.";
      usernameInput.focus();
    }
  });
</script>
```

Summary: Attaching a listener to the submit event allows interception via event.preventDefault(). Using .trim() prevents users from submitting blank spaces.


NOVA
Start building on
Nova Cloud
Architecture
Create your developer identity to deploy microservices, provision edge clusters, and manage serverless data pipelines globally.
Instant Provisioning
$300 Free Cloud Credits
SOC2 Type II Certified
Create Developer Account
Full Name *
Work Email Address *
Cloud Username *
Password *
Confirm Password *
Complete Cloud Registration

Question 9: Interactive Welcome Message Button

Implementation:

HTML

```
<button id="welcomeBtn">Start Learning</button>
<p id="msgDisplay"></p>
<script>
  const welcomeBtn = document.getElementById('welcomeBtn');
  const msgDisplay = document.getElementById('msgDisplay');

  welcomeBtn.addEventListener('click', () => {
    msgDisplay.textContent = "Welcome to your e-learning dashboard! Happy studying.";
    msgDisplay.style.color = "#2b8a3e";
  });
</script>
```

Summary: Uses document.getElementById to target the button and message paragraph, binding an event listener for 'click' that updates textContent dynamically without refreshing the page.



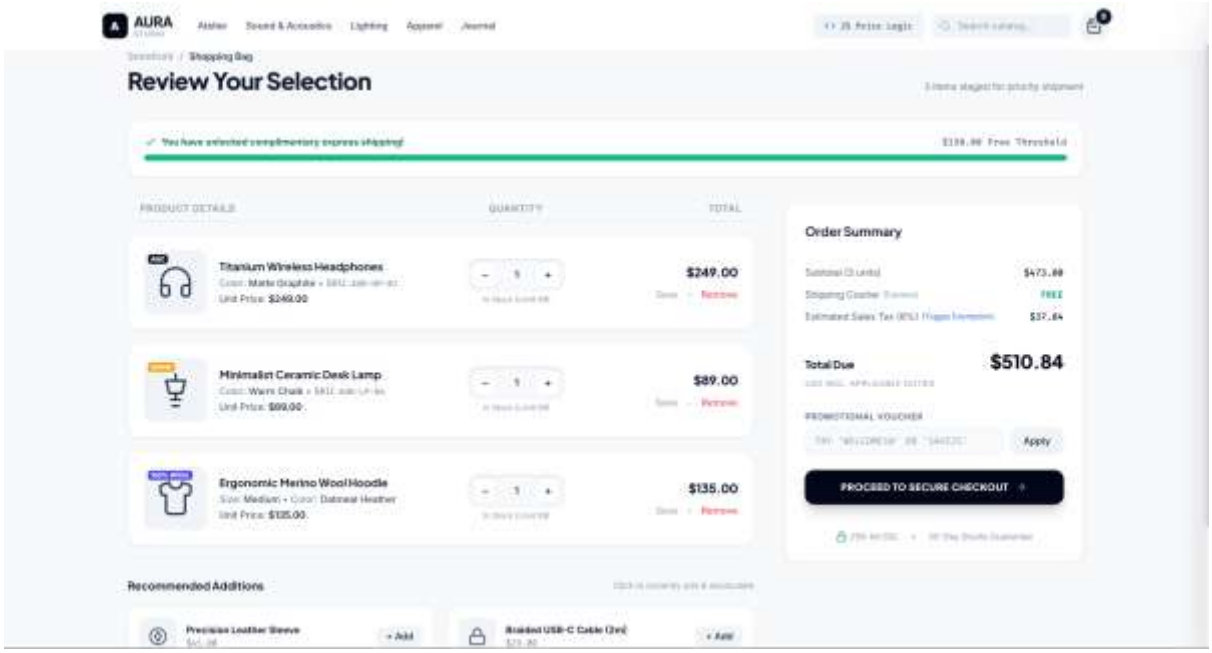
Question 10: Dynamic Total Price Calculation Based on Quantity

Implementation:

HTML

```
<div class="checkout-item">
  <p>Unit Price: $<span id="unitPrice">25</span></p>
  <label for="quantity">Quantity:</label>
  <input type="number" id="quantity" value="1" min="1">
  <h3>Total Price: $<span id="totalPrice">25</span></h3>
</div>
<script>
  const unitPrice = parseFloat(document.getElementById('unitPrice').textContent);
  const qtyInput = document.getElementById('quantity');
  const totalDisplay = document.getElementById('totalPrice');
  function calculateTotal() {
    let quantity = parseInt(qtyInput.value, 10);
    // Safeguard against negative or NaN values
    if (isNaN(quantity) || quantity < 1) {
      quantity = 1;
      qtyInput.value = 1;
    }
    const total = (unitPrice * quantity).toFixed(2);
    totalDisplay.textContent = total;
  }
  // Calculate instantly upon typing or stepper change
  qtyInput.addEventListener('input', calculateTotal);
</script>
```

Summary: The input event listener fires whenever the numeric input changes, calculating unitPrice * quantity and updating the #totalPrice span immediately.


AURA
Atelier
Sound & Acoustics
Lighting
Apparel
Journal
Shopping Bag
Review Your Selection
Titanium Wireless Headphones
Unit Price: $249.00
$249.00
Minimalist Ceramic Desk Lamp
Unit Price: $89.00
$89.00
Ergonomic Merino Wool Hoodie
Unit Price: $135.00
$135.00
Order Summary
Total Due
$510.84
PROMOTIONAL VOUCHER
Apply
PROCEED TO SECURE CHECKOUT
Recommended Additions